

Assignment 2 : Null-Safe Async Fetcher

Name: Parth Sahani

Cohort: Elon Musk

Roll Number: 150096724135

Github Fork Link:

https://github.com/Indianworldruler/parth_assignments/

Please do check the branches for all the submitted assignments

Explanation at the last of the screenshots

PLEASE TURN OVER

Screenshots:

```
● (base) parthsahani@Parths-MacBook-Air Assignment2 % dart run Async_Programming.dart
===== MOCK API DATA =====

API: Fetching user profile...
API: User data loading...
User data: Parth data received
Email is not available
Data checking completed

Program finished
❏ (base) parthsahani@Parths-MacBook-Air Assignment2 %
```

What I Learned From The Null Safety And Async Programming Assignment

1. Understanding Future

- I learned that Future is used when a result is not available immediately.
- It is mostly useful for tasks which take some time like getting data from an API downloading a file or loading some information.
- In my program I used Future<String> for the mock API function.

The String part means that when the Future finishes it will return a String value.

For example Future<String> getUserData() means the function will give a String in the future. It does not give the result immediately.

I understood this with the example of an API. When an application asks an API for data the data may take some time to arrive. So instead of stopping the complete program we can use Future to handle this waiting time.

2. Understanding async

- I learned that async is used with a function which performs asynchronous work.
- In my program getUserData() is an async function.
- I also used async in showUserData() and main().

When a function is marked with async it can use await inside it.

For example `Future<String> getUserData()` `async` means the function returns a `Future` and it is able to wait for asynchronous operations.

Before this assignment I mostly used normal functions where the result comes directly. Now I understood that some operations need time so Dart provides asynchronous programming for these situations.

3. Understanding `await`

- I learned that `await` is used to wait for a `Future` to finish.
- In my program I used `String user = await getUserData();`
- This means Dart waits until `getUserData()` finishes. After the result comes the value is stored inside the `user` variable.

I understood that `await` makes asynchronous code easier to read because it looks similar to normal step by step code.

Another example in my program is `await Future.delayed(Duration(seconds: 2));`

This waits for two seconds before returning the mock API data.

The important thing I understood is that `await` is normally used inside an `async` function.

4. `Future.delayed()`

- I learned how `Future.delayed()` can be used to simulate something taking time.
- I used two seconds delay in my mock API.

The code `await Future.delayed(Duration(seconds: 2));` is acting like the application is waiting for an API response.

There is no real API in my assignment. The function is only pretending to fetch data. This helped me understand asynchronous programming without needing a backend or actual internet API.

5. Mock API

- I learned what a mock API means in a simple way.
- A mock API is basically a fake API used for testing or learning.
- In my program `getUserData()` acts like the API.

It prints that user data is loading then waits for two seconds. After that it returns the message "Parth data received".

So the program behaves like it requested some data from a server.

I understood that in a real application the API would normally return data from a server. Here I only used a normal Dart function to understand the concept.

6. Understanding Null Safety

- I learned that Dart has null safety to reduce errors caused by missing values.
- A normal `String` cannot contain null.
- If I want a `String` to contain null I have to write `String?`.

In my program I used `String? getUserEmail(bool hasEmail)`.

The `?` means the returned `String` can either contain a value or be null.

For example if the user has an email the function returns the email. If there is no email it returns `null`.

This helped me understand why nullable variables are useful. Sometimes information may not be available so the program should handle that situation instead of crashing.

7. Null Check

- I learned how to check if a nullable value actually contains something.
- I used `if (email != null)`.

If the email is not null the program displays it. If it is null the program displays that the email is not available.

This is a simple way of handling missing data.

I understood that we should not directly assume that a nullable variable has a value. First we should check it.

8. try and catch

- I learned how `try` and `catch` are used for error handling.
- Code which may cause an error is placed inside `try`.
- If an error happens the `catch` block handles it.

In my program I used `try` for the operation which could have an error. Then I used `catch (error)` to handle the problem.

The `catch` block prints the error.

This is useful for API related operations because sometimes an API may fail or some unexpected problem can happen.

Instead of the whole program stopping suddenly we can handle the problem properly.

9. finally

- I also learned about the `finally` block.
- It runs after the `try` or `catch` part is completed.
- It runs whether an error happens or not.

In my program I used `finally` to print "Data checking completed".

So even if the API operation gives an error the final message can still be printed.

I understood `finally` as a part which is used for something that should happen at the end of the operation.

How I Combined The Concepts

The main thing I learned from this assignment was how all these concepts work together instead of only using them separately.

I first created `getUserData()` which acts as a mock API. I made it return `Future<String>` because the data is supposed to arrive later. I used `async` so that I could use `await` inside the function.

Then I used `Future.delayed()` to create a small waiting time. After two seconds the function returns the user data.

After that I created `getUserEmail()`. This function demonstrates null safety. I used `String?` because the email may not be available.

Then I created `showUserData()`. This is where most of the concepts are connected. I used `try` for the operation which could have an error. I used `await` to wait for the mock API. After receiving the user data I checked the email using `email != null`.

If the email is available the program displays it. If it is missing the program displays that the email is not available.

Finally I used `finally` so that the completion message is shown at the end.

The complete flow of my program is:

`main()` → `showUserData()` → `getUserData()` → `Future.delayed()` →
API data → null check → display result → finally

This made the complete concept much easier for me to understand.

What I Understood

- `Future` is used when a result will come later.
- `async` allows a function to perform asynchronous operations.
- `await` waits for a `Future` to complete.
- `Future.delayed()` can be used to create a delay.
- A mock API can be used to practise API concepts without a real server.
- `String?` means the `String` can have a null value.
- `if (value != null)` checks whether the value exists.

- `try` is used for code which may cause an error.
- `catch` handles the error.
- `finally` runs at the end whether an error happens or not.
- `main()` starts the program.

I also understood that asynchronous programming is important because applications often have to wait for something. For example an app may need to load user data from a server. If we handle this properly the application can continue working instead of getting stuck.

Problems I Faced While Doing The Assignment

- One basic problem I faced was understanding the difference between `Future` and a normal return value.
- At first I was confused why the function needs to return `Future<String>` instead of just `String`.
- I also had some confusion about where `async` should be written.
- Another problem was understanding that `await` can only be used properly inside an `async` function.
- I sometimes forgot that the program has to wait for the `Future` before using its result.
- Null safety was also confusing in the beginning because `String` and `String?` look almost same.
- I had to understand why `?` is required when a value can be null.
- I also faced small syntax mistakes while writing `try catch` and `finally`.
- Sometimes I forgot to put the correct brackets in the asynchronous functions.
- Another small problem was understanding the order in which the functions were running.
- I had to read the output carefully to understand when the `Future` completed.
- Since this was a mock API I also had to understand that the delay was only being used to simulate real API behaviour.